B9AI103 Recommender Systems

CA ONE Group Report

Module Code: B9AI103

Module Title: Recommender Systems

Lecturer: Nitya Govindaraju

Group Members

Tawanda Banda 20079638

Bhargava Koya 20075522

Chandra Binod 20079103

Surya Dharmaprakash 20076570

1. Introduction

The modern job market generates millions of postings daily, making it increasingly difficult for candidates to identify roles that match their skills, experience, and preferences. Recommender systems address this information overload by automatically surfacing the most relevant items from a large catalogue, reducing search friction and improving match quality for both candidates and employers.

This report documents the design, implementation, and evaluation of two distinct job recommender systems built on the LinkedIn Job Postings dataset a real-world corpus of approximately 128325 job listings spanning 31 attributes. The two systems take fundamentally different approaches to similarity:

- System 1 uses TF-IDF vectorisation combined with structured feature scoring (experience level, location, salary) in a weighted hybrid model.
- System 2 uses Word2Vec embeddings trained on job descriptions, capturing semantic similarity rather than keyword overlap.

Both systems are evaluated using standard offline ranking metrics NDCG and MAP and a cold-start mitigation strategy is implemented and demonstrated. The report concludes with an analysis of how Generative AI approaches, specifically Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs), could augment or replace the implemented systems.

2. Dataset & Exploratory Data Analysis

The LinkedIn Job Postings dataset (Kaggle) contains 128325 job listings with 31 columns including job title, company name, description, required skills, location, experience level, work type, normalised salary, view count, and application count. Due to computational constraints on Google Colab, a stratified sample of 50,000 records was used for development and evaluation.

Key findings from exploratory data analysis:

- Missing values were significant in skills_desc (~98%) and normalized_salary (~38%), requiring imputation strategies before modelling.
- The most common experience levels were Mid-Senior and Entry level, with Executive roles underrepresented reflecting the general composition of the LinkedIn platform.
- California (CA), New York (NY), and Texas (TX) accounted for the majority of listings, consistent with the concentration of major employers in these states.
- Salary distribution was right-skewed; the top 1% of salaries were removed as outliers prior to min-max normalisation.
- The most frequent job titles Software Engineer, Data Scientist, and Registered Nurse reflected the dataset's professional focus and the dominance of tech and healthcare.

3. System 1 Content-Based Filtering (TF-IDF + Hybrid Scoring)

System 1 is a content-based recommender that measures similarity between jobs using both textual content and structured attributes. The rationale for content-based filtering is its independence from user interaction history, making it immediately applicable to any job posting in the dataset including newly listed roles, which provides a natural partial solution to the cold-start problem.

3.1 Preprocessing & Feature Engineering

Three text columns were combined into a single content field using a weighted concatenation strategy designed to amplify the most informative signals:

content = title (x3) + description (x1)

Job title was repeated three times because it is the single most discriminating signal a "Data Scientist" posting and a "Nurse" posting share almost no relevant vocabulary, and title similarity is the strongest proxy for job-level relevance. Skills were repeated twice as they are more specific than free-text descriptions, which often contain generic company boilerplate. All text was lowercased, HTML tags and URLs were removed, non-alphabetic characters were stripped, and whitespace was normalised.

Three structured features were also encoded for use in hybrid scoring:

Feature	Encoding	Rationale
Experience Level	Ordinal integer (Internship=0 to Executive=5), normalised 0-1	Preserves natural ordering of seniority; missing values filled with median
Location	Extract 2-letter state code; binary state-match signal	State-level proximity as a practical filter for commutable roles
Salary	Min-max scaled 0-1; missing values filled with dataset median	Comparable scale for gap-based similarity scoring

3.2 TF-IDF Vectorisation

Term Frequency-Inverse Document Frequency (TF-IDF) converts the content field into a sparse numerical matrix where each value reflects how important a word is within a specific job relative to the entire corpus. Key configuration decisions:

- max_features=15,000 retains the 15K most informative tokens, balancing vocabulary coverage with computational efficiency on a 50K-document corpus.
- ngram_range=(1,2) includes both single words and two-word phrases (e.g. "machine learning", "project manager"), capturing compound concepts that single tokens miss.
- min_df=2, max_df=0.85 excludes tokens that are either too rare (noise) or too common across documents (uninformative, stopword-like terms beyond the English list).
- sublinear_tf=True applies log-normalisation to term frequency, reducing the outsized influence of repeated terms within long job descriptions.

3.3 Hybrid Recommendation Engine

The final similarity score combines TF-IDF cosine similarity with four structured feature similarities using a configurable weighted sum:

hybrid_score = 0.60 x text_sim
+ 0.15 x experience_sim
+ 0.15 x location_sim
+ 0.10 x salary_sim

Text similarity dominates at 60% because it captures the broadest and most informative signal, job role and required skills. Experience and location are weighted equally at 15% as they are both strong practical filters for candidates. Salary is weighted lowest at 10% because salary data is missing for approximately 38% of postings and salary preferences are highly individual. All weights are

configurable per query; a weight sensitivity analysis in the notebook demonstrates how shifting emphasis towards location or salary changes which jobs are surfaced.

The system supports three query modes: by job index (item-to-item), by title keyword (finds the first matching job and recommends similar ones), and by full user profile (skills list, experience level, target state, and salary target).

4. System 2 Embedding-Based Filtering (Word2Vec)

System 1's primary limitation is that TF-IDF treats each word as an independent feature with no understanding of meaning. "Software engineer" and "developer" would score zero cosine similarity despite being functionally synonymous. System 2 addresses this by replacing the sparse TF-IDF matrix with dense semantic embeddings trained on the job corpus using Word2Vec (Mikolov et al., 2013).

4.1 Model Training

A Word2Vec model was trained directly on the 50,000 job postings using the same pre-cleaned content field from Section 3.1. Domain-specific training is important: generic pre-trained embeddings (e.g. Google News Word2Vec) would not capture the specific meaning of technical terms and job-specific vocabulary in context. Configuration:

Parameter	Value	Rationale
vector_size	100	Balances representational capacity with memory efficiency
window	5	Words within +/-5 positions considered contextually related
min_count	2	Filters noise from tokens appearing only once in the corpus
epochs	5	Sufficient convergence for a 50K-document training corpus

4.2 Job Vector Construction

Each job posting is represented as a single 100-dimensional vector by averaging the Word2Vec vectors of all tokens in its content field that appear in the model's vocabulary. Averaging is a simple but effective aggregation method for document-level embeddings; it is computationally inexpensive and works well when individual word vectors are of high quality. Jobs whose entire content falls outside the vocabulary receive a zero vector and naturally rank lowest in similarity searches.

4.3 Recommendation Engine

Given a query job index, the cosine similarity between its embedding and all other job embeddings is computed vectorially. The top-N highest-scoring jobs (excluding the query itself) are returned as recommendations. Unlike System 1, System 2 operates purely on semantic content without structured feature weighting, making the two systems genuinely complementary: System 1 excels at precise keyword matching with structured filtering; System 2 generalises across roles described with different vocabulary but equivalent meaning.

5. Evaluation

5.1 Evaluation Strategy & Surrogate Labels

The dataset contains no explicit user-item interaction data (e.g. ratings or click histories), making standard supervised evaluation impossible. An offline surrogate evaluation strategy was adopted, consistent with common practice in the literature for content-based recommender evaluation without interaction logs.

Relevance labels were derived from title similarity: a recommended job is considered relevant if its title shares at least one keyword of four or more characters with the query job's title. This operationalises the intuition that a good recommender should surface jobs in the same broad role category as the query. Both systems were evaluated on 200 randomly selected query jobs (seed=42) with top-10 recommendations.

5.2 Metrics

Metric	Description	What It Measures
NDCG@10	Normalised Discounted Cumulative Gain at 10	Ranking quality — rewards placing relevant items higher; normalised against ideal ordering
MAP@10	Mean Average Precision at 10	Precision computed at each relevant position, averaged — penalises late discoveries
Title Match Rate@10	Fraction of top-10 recs sharing a title keyword	Proxy precision — category coherence of recommendations
Intra-list Diversity@10	Average pairwise cosine distance among top-10 recs	Diversity — avoids filter bubbles within a single recommendation set

5.3 Results

Results below show the metric values as produced when the notebook cells are executed in full. Actual values will reflect the sampled 50,000 rows and the trained Word2Vec model weights.

Metric	System 1 (TF-IDF)	System 2 (Word2Vec)
NDCG@10	[run notebook — Section 10]	[run notebook — Section 10]
MAP@10	[run notebook — Section 10]	[run notebook — Section 10]
Title Match Rate@10	[run notebook — Section 8]	—
Intra-list Diversity	[run notebook — Section 8]	—

An important limitation of the surrogate evaluation is that it inherently favours System 1: because TF-IDF directly matches job title keywords, it trivially recovers jobs with identical titles. System 2, by contrast, may recommend semantically related but differently titled roles (e.g. "Data Analyst" for a "Business Intelligence Engineer" query) that would be genuinely useful to a candidate but score zero under the title-keyword surrogate. This reflects the broader challenge of offline evaluation of content-based systems without ground-truth interaction data.

6. Cold-Start Mitigation

6.1 Problem Definition

The cold-start problem arises when a recommender system lacks sufficient information to generate reliable recommendations. It manifests in two distinct scenarios in the job recommendation domain:

- New user cold-start: A candidate using the platform for the first time has no interaction history (no applied jobs, no viewed listings), so the system cannot personalise based on past behaviour.
- New item cold-start: A job posting published minutes ago has zero views and zero applications, so engagement-based signals are unavailable for ranking.

Collaborative filtering approaches suffer severely from both scenarios. Content-based systems are inherently more resilient to new item cold-start because a job's text content is available the moment it is posted making Systems 1 and 2 naturally better suited to this dataset.

6.2 Strategy: Popularity-Based Fallback (New Item)

For new items with no engagement data, a popularity-based fallback was implemented. Each job is assigned a popularity score computed as a weighted combination of view count (40%) and application count (60%), normalised to the range [0, 1]:

popularity_score = 0.4 x views_norm + 0.6 x applies_norm

Applications are weighted more heavily than views because they represent a stronger signal of genuine candidate interest. When engagement data is absent, the system falls back to this popularity ranking, optionally filtered by the user's stated experience level and preferred state. If the filtered result set is smaller than the requested number of recommendations, the filter is relaxed to the global popularity ranking, ensuring the user always receives a complete list.

6.3 Strategy: Profile-Based Onboarding (New User)

For new users with no history, a profile-based onboarding flow was implemented via the get_recommendations_by_profile() function. The user provides a list of skills or keywords, their experience level, a preferred state, and a target salary. These inputs construct a synthetic query: the skills list is transformed by the TF-IDF vectoriser, and the structured features are encoded identically to existing jobs. The hybrid scoring function then ranks all jobs in the catalogue against this synthetic profile, providing a personalised recommendation list without any prior interaction history.

This strategy is effective because content-based filtering does not require historical interactions it only requires a description of what the user is looking for, which can be collected in a brief onboarding questionnaire. As the user engages with the platform, interaction history can progressively refine and override these profile-based recommendations.

7. Generative AI Analysis

Generative AI models represent a fundamentally different paradigm for recommender systems. Rather than measuring similarity between existing representations, they learn to generate plausible user preferences or item features from latent distributions enabling richer personalisation, better uncertainty modelling, and more effective handling of data sparsity.

7.1 Role of VAEs in Recommender Systems

Variational Autoencoders (VAEs) are generative models that encode input data into a compressed probabilistic latent space and decode back to reconstruct the input. In recommender systems, the input is typically a user's interaction history a sparse vector of item ratings or click events.

The most influential application is Mult-VAE (Liang et al., 2018), which applies a multinomial likelihood function suited to implicit feedback (clicks, views, applies) rather than explicit ratings. Mult-VAE has been shown to outperform classical matrix factorisation methods (e.g. Weighted Matrix Factorisation) on large-scale collaborative filtering benchmarks, particularly on sparse datasets. The probabilistic latent space allows the model to express uncertainty about a user's preferences and generate diverse, non-trivial recommendations rather than defaulting to popular items.

7.2 Role of GANs in Recommender Systems

Generative Adversarial Networks consist of two networks a generator that proposes candidate outputs and a discriminator that evaluates their authenticity trained adversarially until the generator produces outputs indistinguishable from real data. In recommender systems:

- CFGAN (Chae et al., 2018) applies this framework to collaborative filtering by training a generator to produce realistic user interaction vectors. The discriminator learns to distinguish generated from real interaction patterns, forcing the generator to accurately model true preference distributions.
- RecGAN uses the generator as a data augmentation tool synthesizing plausible interaction data for sparse users and new items, directly addressing both forms of cold-start without requiring large amounts of real interaction data.

7.3 Comparison to Implemented Systems

Aspect	System 1 (TF-IDF)	System 2 (Word2Vec)	Generative AI (VAE/GAN)
Learns from interactions	No	No	Yes
Semantic understanding	Low (keyword)	Medium (context)	High (latent space)
Cold-start handling	Profile fallback	Profile fallback	Data augmentation via GAN
Scalability	High	Medium	Low to Medium
Explainability	High	Medium	Low (black box)
Data requirement	Text only	Text only	Interaction history needed

7.4 Potential Enhancements to This System

Three concrete ways Generative AI could enhance the implemented job recommender:

- Mult-VAE for interaction-based personalisation: If augmented with per-user interaction logs, a Mult-VAE could learn dense user embeddings from engagement history capturing latent preferences that keyword and embedding similarity cannot, such as a consistent preference for remote roles even when location is unspecified in search queries.
- CFGAN for cold-start augmentation: For newly posted jobs with zero engagement, a GAN trained on existing job-engagement patterns could synthesise realistic interaction vectors, bootstrapping the item into the collaborative filtering space before real interactions accumulate. This would remove the need for the popularity-based fallback.

- Conditional VAE for profile-driven generation: A CVAE conditioned on user profile attributes (skills vector, experience level, salary range) could generate a personalised job feature distribution characterising what the ideal job for this user looks like and matching it against the catalogue, rather than simply returning the most similar existing jobs.

The primary barrier to deploying these approaches on the current dataset is the absence of per-user interaction histories. The dataset provides aggregate view and apply counts at the job level but does not link individual users to individual interactions, which is a prerequisite for all three generative approaches. Additionally, GANs introduce training instability risks and VAEs can suffer from posterior collapse both requiring careful hyperparameter tuning that is non-trivial without extensive compute resources.

8. Conclusion

This project designed and implemented two complementary job recommender systems on the LinkedIn Job Postings dataset. System 1 combines TF-IDF keyword matching with structured feature similarity in a configurable hybrid model, offering high precision and full interpretability. System 2 uses Word2Vec embeddings to capture semantic similarity, bridging the vocabulary gap that limits bag-of-words approaches.

Both systems were evaluated offline using NDCG@10 and MAP@10 with surrogate relevance labels derived from job title similarity, alongside Title Match Rate and Intra-list Diversity as supplementary metrics. The evaluation highlights a key limitation of surrogate-label evaluation: it inherently favours keyword-based systems like System 1, even though System 2 may provide more genuinely useful cross-role recommendations in practice.

Cold-start mitigation was addressed through two complementary strategies: a popularity-based fallback weighted by views and applications for new items, and a profile-based onboarding flow for new users. Both strategies are immediately deployable without any interaction history, leveraging the natural resilience of content-based filtering to the cold-start problem.

The Generative AI analysis demonstrates that VAE and GAN-based approaches in particular Mult-VAE, CFGAN, and conditional VAEs could substantially improve personalisation and cold-start handling given per-user interaction data. The current dataset's lack of user-level logs is the primary constraint preventing direct application of these techniques, and represents a clear direction for future work.

References

1. Ber, L. (2023). *LinkedIn Job Postings — 2023.* Kaggle dataset. Retrieved from https://www.kaggle.com/datasets/arshkon/linkedin-job-postings
2. Chae, D. K., Kang, J. S., Kim, S. W., & Lee, J. T. (2018). *CFGAN: A generic collaborative filtering framework based on generative adversarial networks.* In Proceedings of the 27th ACM International Conference on Information and Knowledge Management (pp. 137-146). ACM.
3. Järvelin, K., & Kekäläinen, J. (2002). *Cumulated gain-based evaluation of IR techniques.* ACM Transactions on Information Systems, 20(4), 422-446.

4. Liang, D., Krishnan, R. G., Hoffman, M. D., & Jebara, T. (2018). *Variational autoencoders for collaborative filtering.* In Proceedings of the 2018 World Wide Web Conference (pp. 689-698). ACM.
5. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space.* arXiv preprint arXiv:1301.3781.
6. Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python.* Journal of Machine Learning Research, 12, 2825-2830.
7. Rehůrek, R., & Sojka, P. (2010). *Software framework for topic modelling with large corpora.* In Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks (pp. 45-50).
8. Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). *Recommender systems handbook* (2nd ed.). Springer.